

Module 6 Week A — Applied Lab: NER Pipeline

In this lab you will build a Named Entity Recognition pipeline on a real climate articles dataset. You will preprocess text, extract entities using both spaCy and Hugging Face, compare their outputs, evaluate NER quality against a gold standard, and run a multilingual model on the Arabic subset of the corpus. By the end, you will have a working NER system that extracts structured facts from unstructured text — and you will have direct evidence that a single-language model is not enough when your data spans languages.

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

Learning Objectives

- Load and explore a multilingual text dataset
- Write a text preprocessing function with Unicode normalization and lemmatization
- Extract named entities using spaCy's built-in NER
- Extract named entities using Hugging Face's NER pipeline
- Compare entity outputs from two different NER systems
- Evaluate NER quality with entity-level precision, recall, and F1
- Apply a multilingual NER model to Arabic text and observe how it differs from the English-only models

Setup

1. **Accept the assignment**
2. **Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

The Hugging Face NER pipeline uses PyTorch. `torch` is listed in `requirements.txt` (version 2.2 or higher), and `pip install` will download the correct version for your computer automatically. Intel Macs need 2.2.x because PyTorch dropped x86_64 macOS wheels at 2.4; Apple Silicon and Linux learners pick up a newer release.

4. **Download the spaCy English model:**

```
python -m spacy download en_core_web_sm
```

5. **Download the spaCy multilingual model** (used in Task 7 for Arabic):

```
python -m spacy download xx_ent_wiki_sm
```

This is a small (~11 MB) multilingual NER model trained on Wikipedia. Unlike `en_core_web_sm` (English only), it can process Arabic, French, Chinese, and many other languages.

6. **Create a working branch:**

```
git checkout -b lab-6a-ner-pipeline
```

The starter file `ner_pipeline.py` contains function signatures with docstrings and TODO comments. The dataset is at `data/climate_articles.csv` and the gold standard annotations are at `data/gold_entities.csv`. Read the TODO directions, consult the Week A reading, and implement each function.

Note: The first time you run the Hugging Face NER pipeline, it will download model weights (~250 MB for `dslim/bert-base-NER`). This is a one-time download. Run `python -c "from transformers import pipeline; pipeline('ner', model='dslim/bert-base-NER')"` before starting the lab to trigger the download.

The Dataset

`climate_articles.csv` contains ~200 climate-related text snippets. Each row has:

Column	Type	Description
<code>id</code>	int	Unique text identifier
<code>text</code>	str	Climate article text (~35–75 words per snippet)
<code>source</code>	str	Source publication
<code>language</code>	str	Language code: <code>en</code> (English) or <code>ar</code> (Arabic)
<code>category</code>	str	Content category: <code>policy</code> , <code>science</code> , <code>impact</code> , or <code>adaptation</code>

The texts reference organizations (IPCC, UNEP, COP28), locations (Jordan, Dead Sea, Arctic), people (by role, not name), dates, and quantities.

`gold_entities.csv` contains manually annotated entities for 10 of the texts:

Column	Type	Description
<code>text_id</code>	int	References <code>id</code> in <code>climate_articles.csv</code>
<code>entity_text</code>	str	The entity as it appears in the text
<code>entity_label</code>	str	Entity type (ORG, GPE, DATE, etc.)
<code>start_char</code>	int	Start character offset
<code>end_char</code>	int	End character offset

Note on the gold standard size: Only 10 texts are annotated (roughly 69 entities total). Treat precision/recall/F1 numbers you compute against this set as indicative, not statistically robust — focus on the pattern of errors (which categories or entity types each system misses) rather than the absolute metric values.

Task 1: Load and Explore the Data

Implement two functions:

- `load_data(filepath)` — read `data/climate_articles.csv` into a pandas `DataFrame` and return it.
- `explore_data(df)` — return a dictionary with the following keys:
 - `shape`: a `(n_rows, n_cols)` tuple
 - `lang_counts`: a dict mapping language code (`en`, `ar`) to row count
 - `category_counts`: a dict mapping each category (`policy`, `science`, `impact`, `adaptation`) to row count
 - `text_length_stats`: a dict with `mean`, `min`, and `max` word counts across `df['text']`

Confirming the shape and distributions before processing catches data surprises early (unexpected languages, missing categories, empty texts) — a habit every NLP pipeline needs.

Task 2: Preprocess Text

Implement `preprocess_text(text, nlp)`:

- Apply Unicode normalization (NFC) to the input string
- Pass the normalized text through the injected `nlp` pipeline
- Iterate over the resulting tokens, dropping those that are punctuation (`token.is_punct`) or whitespace (`token.is_space`)
- Return the lowercased lemma (`token.lemma.lower()`) for each surviving token

The `nlp` object is loaded **once** in `__main__` and passed into every function that needs it — this is the standard spaCy pattern (the model is expensive to load; never call `spacy.load()` inside a per-text function). All subsequent tasks follow the same injection pattern.

Test your function on a sample text before applying it to the full dataset. Verify that the output is lowercase, punctuation-free, and lemmatized (e.g., "released" becomes "release", "policies" becomes "policy").

Task 3: spaCy NER Extraction

Implement `extract_spacy_entities(df, nlp)`:

- Filter the `DataFrame` to English-language texts only (`language == 'en'`) — spaCy's `en_core_web_sm` is English-only, and Arabic text produces noise
- Process each English text through `nlp(text)` using the injected pipeline
- Extract all named entities from `doc.ents`
- Return a `DataFrame` with columns: `text_id`, `entity_text`, `entity_label`, `start_char`, `end_char`

Each row in the output `DataFrame` represents one entity found in one text. Include all entity types that spaCy finds.

Task 4: Hugging Face NER Extraction

Implement `extract_hf_entities(df, ner_pipeline)`. Hugging Face (HF) is the library ecosystem the `transformers` package belongs to; the rest of this guide uses "HF" as shorthand.

- Filter the `DataFrame` to English-language texts only
- Run each English text through the injected `ner_pipeline` (`dslim/bert-base-NER`, loaded once in `__main__`)
- Clean up the raw output in two steps (see the reading, Section 8):
 1. **Merge subword tokens.** BERT uses WordPiece tokenization: "IPCC" comes back as `IP` + `##CC`. Merge continuation tokens (those starting with `##`) back into complete entity text.
 2. **Strip the IOB label prefix.** HF NER emits labels in **IOB format** (Inside / Outside / Beginning):
B-ORG marks the beginning of an ORG span. I-ORG marks a continuation. After merging, strip the `B-` / `I-` prefix so your labels match spaCy's (`ORG`, `LOC`, `PER`, `MISC`) — the comparison and evaluation tasks downstream depend on this.
- Return a `DataFrame` with the same columns: `text_id`, `entity_text`, `entity_label`, `start_char`, `end_char`

Task 5: Compare NER Outputs

Implement `compare_ner_outputs(spacy_entities_df, hf_entities_df)`:

- Count the total entities found by each system
- Count entities by label for each system (how many ORG, GPE, DATE, etc.)
- Compute three overlap sets by matching on `(text_id, entity_text)`:
 - `both`: entities found by spaCy and HF in the same text
 - `spacy_only`: entities only spaCy found
 - `hf_only`: entities only HF found

Return a dictionary with keys `spacy_counts`, `hf_counts`, `total_spacy`, `total_hf`, `both`, `spacy_only`, `hf_only`. Print a summary showing where the two systems agree and where they diverge.

Task 6: Evaluate Against Gold Standard

Implement `evaluate_ner(predicted_df, gold_df)`:

- The function receives predicted and gold entities as `DataFrames` with columns `text_id`, `entity_text`, `entity_label`. Loading `data/gold_entities.csv` happens in `__main__` and is passed in as `gold_df`.
- **Filter `predicted_df` to the set of `text_ids` in `gold_df` before scoring.** Only 10 of the 132 English texts have gold annotations. Predictions on the other 122 cannot be true positives — counting them as false positives would inflate your false-positive count by ~17% and crush your precision number into the noise.
- For each gold-annotated text, compare the (filtered) predicted entities against the gold entities
- Compute entity-level precision, recall, and F1 (an entity is correct only if both the text and label match exactly)
- Call the function twice from `__main__` — once with the spaCy predictions and once with the Hugging Face predictions — so you can compare metrics across the two systems

Return a dictionary with keys `"precision"`, `"recall"`, and `"f1"` (values must be floats in the 0–1 range).

Task 7: Multilingual NER on Arabic

So far every NER call has been on English text — Tasks 3 and 4 explicitly filter the corpus to `language == 'en'` because `en_core_web_sm` and `dslim/bert-base-NER` are both English-only. The dataset has **68 Arabic rows**. Running the English models on those would produce noise. The right tool for non-English NER is a multilingual model.

Implement `extract_multilingual_entities(df, multilingual_nlp)`:

- Filter `df` to Arabic-language rows (`language == 'ar'`)
- Process each text through the injected `multilingual_nlp` (`xx_ent_wiki_sm`, loaded once in `__main__`)
- Extract entities from `doc.ents`
- Return a `DataFrame` with the same columns as `extract_spacy_entities`: `text_id`, `entity_text`, `entity_label`, `start_char`, `end_char`

The multilingual model uses the **multilingual Wikipedia entity schema** — `ORG`, `PER`, `LOC`, `MISC`. (Note: `PER` and `LOC` here, not `PERSON` and `GPE` like spaCy's English model — different schemas come with different pretraining datasets.)

Quality on Arabic news text will be mediocre. `xx_ent_wiki_sm` is a small model and Arabic NER is a hard task even for large models. Your goal here is to **see what happens when you process non-English text with the right tool, not the wrong one**, and to confirm that a single-language pipeline does not generalize across languages — picking the right model for the data is part of the job.

Challenge Extensions

These are optional extensions for learners who want to go deeper. They are not graded and do not affect your lab score. No starter code is provided — design and implement these independently.

Tier 1: Per-Category NER Analysis

The climate dataset has four categories: `policy`, `science`, `impact`, and `adaptation`. Different categories may contain different entity distributions.

- Compute entity counts by label for each category separately (e.g., `policy` texts may have more ORG and GPE entities; `science` texts may have more DATE and CARDINAL entities).
- Compute NER precision and recall per category using the gold standard (if gold-annotated texts span multiple categories).
- Produce a heatmap or grouped bar chart showing the entity-type-by-category distribution.
- Write a 1-paragraph analysis: which categories are easiest/hardest for NER and why?

Tier 2: Custom Entity Aggregation Pipeline

Build a reusable entity aggregation module that goes beyond simple counting:

- Implement entity normalization: merge variants of the same entity ("United Nations", "UN", "U.N.") into a canonical form using a mapping dictionary you define.
- Compute entity co-occurrence: for each pair of entities that appear in the same text, count the co-occurrence frequency. Represent this as an adjacency matrix or edge list.
- Implement TF-IDF-style entity importance: weight entity frequency by inverse document frequency to find entities that are distinctive to specific categories rather than appearing everywhere.
- Produce a network visualization of the top 20 entity co-occurrences using `matplotlib` (nodes = entities, edges = co-occurrence count).

Tier 3: Build a Custom NER Evaluator

Design and implement a comprehensive NER evaluation framework from scratch:

- Implement three matching strategies: exact match (text + label must match exactly), partial match (overlapping spans count as partial credit), and type-agnostic match (correct span, any label).
- Compute precision, recall, and F1 under each matching strategy. Implement micro-averaging (pool all entities across texts) and macro-averaging (average per-text scores).
- Build a systematic error analysis: categorize every error as boundary error (wrong span), type error (wrong label), missing entity, or spurious entity. Produce an error distribution report.
- Write at least 5 pytest tests for your evaluator covering edge cases: empty predictions, empty gold standard, overlapping entities, and single-token vs. multi-token entities.

Submission

Open a pull request from `lab-6a-ner-pipeline` to `main`. Your PR description must include:

1. Entity count summary table (spaCy vs. Hugging Face by entity type) on the English subset
2. Precision, recall, and F1 comparison on the gold-annotated subset
3. Brief analysis: where did each system excel or fail? (3–5 sentences). Include one sentence on the Arabic multilingual run — what label set you got, roughly how many entities, and how that differs from the English models.
4. Paste your PR URL into TalentLMS -> Module 6 Week A -> Lab 6A to submit this assignment.

